

M1 Программная основа робота

M1 • 11.10.2026 - 03.11.2026 • 75 часов
Python, C++ и Rust: корпус, один сустав, наблюдение и расследование сбоев

Итог: telemetry-core v1.0. Один репозиторий: единая структура данных IMU и сустава, три утилиты командной строки (CLI), анализатор на Python и основной обработчик на C++. Генератор создаёт данные; стенд показывает крен и ошибку угла, выявляет неверные строки и прекращение передачи, сохраняет журнал для повторения опыта. Это подготовка к ногам и пальцам; приводы и физическая симуляция пока не нужны.

Блок	Даты 2026	Разбор	Практика	Результат
1	11-16 октября	10 ч	10 ч	Среда, снимок и Python
2	18-23 октября	10 ч	10 ч	C++ и регистратор
3	25-30 октября	10 ч	10 ч	Сбои, тесты и Rust
4	1-3 ноября	5 ч	10 ч	Поток, таймаут, воспроизведение
Всего	M1	35 ч	40 ч	75 часов

Календарь. Четыре воскресенья по 10 ч и 17 будней по 2 ч дают 74 ч. Один из 12 дополнительных часов программы назначен на **понедельник 2 ноября: 3 ч**; исходник не задаёт дату этого часа. Субботы свободны. В исходнике M2 также начинается 3 ноября без почасовой границы; его календарь не пересчитывается. Подробные даты и задания - стр. 11-12.

Рабочий ритм и границы

В будни — 120 мин: восстановление контекста 10, чтение 45, пример 35, объяснение 20, запись результата 10. В воскресенье — пять блоков по 2 ч: критерий готовности 10, реализация 90, проверка 20 мин. Перерывы по 15-30 мин и обед не входят в 75 ч. Чтение, словарь и настройка входят.

Веди одну задачу на доске «Следующая / В работе / Проверено». Журнал: дата, минуты, сценарий, начальное значение генератора случайных чисел (seed), оси, единицы, commit, параметры и команда. После 30 мин без продвижения упрости пример; после 60 мин опиши проблему и смени источник. В конце блока обнови README. Основной обработчик — на C++; сокращай расширения раньше базовых тестов.

Приёмка: доказательства, которые нужно сохранить

Подтверди результаты схемой осей и нуля сустава, записями покоя и крена 15°, графиками q_ref и q, событием NO_RESPONSE без догадок о причине. Проверь три CLI и отдельный учёт пропусков, rejected, dropped. Watchdog должен сработать при молчании; дубликат не обновляет таймер, FAULT не сбрасывается сам. Нужны воспроизведение исходных записей; GDB, ASan/UBSan, TSan либо его ограничения; чистая сборка Ubuntu; замеры времени, медианы CLI, p50/p95/p99; размеры очереди и замедленная запись журнала.

Сохрани шесть сценариев, ручные ожидания, графики, события, отчёт и команды. Подсчитай часы, назови три навыка, две ошибки и незавершённые задачи; после помощи AI повтори опыт сам. Несколько суставов потребуют имён и согласованного времени. Балансирование, кинематика, оценка состояния, физические ограничения и управление моментом изучаются дальше.

Основы движения обязательны с M1: звено, сустав, степень свободы, конфигурация и состояние; различие кинематики, динамики и управления. Разбор на стр. 13, расчёт первого механизма на стр. 15. Он входит в блок «оси, единицы, RobotState» 11 октября.

Стенд корпуса и одного сустава

Единый снимок состояния: от чтения полей к наблюдению за роботом

Устройство и координаты

Есть условный корпус с IMU и один вращательный сустав: например, колено ноги или сустав пальца. Используем показания акселерометра IMU, то есть удельную силу. Его оси совпадают с осями корпуса: x вперёд, y влево, z вверх; система правая. Удельную силу измеряем в м/с^2 , угол сустава — в радианах, ток — в амперах. Нуль и положительное направление сустава нарисуй и подпиши в README.

Снимок содержит данные одного шага генератора. У всех полей общая временная метка `t_us`; это упрощение стенда. Реальные IMU и энкодер могут иметь разные частоты и часы. Ток `i_a` здесь лишь записанный сигнал, он не равен моменту привода. `q_ref_rad` - запрошенный угол, `q_rad` - измеренный энкодером.

```
seq,t_us,ax,ay,az,q_ref_rad,q_rad,i_a
0,0,0.0,0.0,9.81,0.0,0.0,0.2
1,10000,0.0,0.0,9.81,0.01,0.008,0.2
```

Контракт данных и ошибок

Используй CSV в UTF-8 без кавычек, с точным заголовком и 8 полями. `seq` и `t_us` - целые числа в записи ASCII $0 \dots 2^{63}-1$. Остальные поля - конечные десятичные числа; допускаются знак, точка и экспонента. Отклоняй hex, подчёркивания, NaN/Inf. Убирай пробелы по краям и проверяй полный разбор. Учебный сустав ограничен интервалом $[-1; 1]$ рад для обоих углов.

Для принятых снимков `seq` и `t_us` строго возрастают. Некорректная строка не обновляет принятое состояние. Разрыв `seq` - диагностическое событие, а не причина отклонять следующий снимок. Сброс счётчика требует начала новой записи; автоматическое распознавание сброса в M1 не нужно.

Проверка	Результат
Заголовок, файл	При неверном заголовке или невозможности чтения прекратить работу, код 2.
Строка или поля	Отклонить: blank, field_count, number, nonfinite, joint_range, sequence или timestamp; это порядок приоритетов.
Итог CLI	Код 0: все строки допустимы; 1: есть отклонённые. JSON в stdout, причины в stderr.
Сводка	rows_total, accepted, rejected, errors_by_reason; first/last_t_us; mean_ax/ay/az; max_abs_joint_error. Пустые метрики: null.

Набор опытов

Создай отдельные файлы по 1 000 снимков с шагом 10 000 мкс: **покой**; **статический крен 15°**; **плавное движение сустава**; **нет отклика после seq=200**; **разрыв seq=40...49**; **плохие строки**. Для реального хода часов используй отдельный потоковый тест с паузой. Проверочные файлы без шума; шум с seed=42 - дополнительный опыт.

Реализации на всех языках читают одни и те же файлы. Ожидаемые результаты малых опытов вычисли вручную. Целые и причины ошибок должны совпадать точно, метрики - с абсолютным допуском 10^{-9} на этих данных. Общая структура: python/, cpp/, rust/, scenarios/, fixtures/, tests/, reports/.

Linux и воспроизводимый опыт

Научиться запускать и расследовать эксперимент с роботом

Какие разделы пройти

- **Файлы и командная оболочка.** Пути, рабочий каталог, права, переменные окружения, кавычки, stdin/stdout/stderr, перенаправления, канал pipe и код завершения. Команды: pwd, ls, cd, mkdir, cp, mv, less, head, tail, rg, chmod, env.
- **Процессы.** PID, работа на переднем плане и в фоне, ps, top, SIGINT/SIGTERM. Разберись, как остановить генератор и дать регистратору закрыть файл; почему kill -9 не позволяет выполнить обычную очистку.
- **Среда Python.** Интерпретатор, виртуальное окружение, pyproject.toml, [uv.lock](https://uv.pypi.org/), uv run и восстановление зависимостей. В проекте нужны numpy, matplotlib и pytest.
- **Git.** Индекс, коммит, diff, ветка и merge; Pro Git 2.1-2.5, 3.1-3.2. Сохраняй историю параметров сценария и изменений в обработке данных датчика.

Настройка рабочего места

На Mac с Apple Silicon используй предусмотренную планом Ubuntu 24.04 ARM64 в UTM для основных запусков. Если Ubuntu уже доступна, работай в ней. Понадобятся терминал, Git, uv/Python, компилятор C++, CMake, GDB, clang-format и Rustup/Cargo. ROS и физический симулятор для этих опытов не требуются.

В README запиши версии, архитектуру и команды. В .gitignore внеси .venv, build, target, кэш и большие результаты. Малые контрольные записи храни в Git; большие создавай генератором. Для каждого опыта сохраняй сценарий, конфигурацию осей и единиц, seed, commit и результат.

Практические задания

L1 Запуск опыта. Сформируй запись неподвижного робота, запусти анализ и отдельно сохрани JSON-сводку и ошибки. Повтори запуск из другого каталога с явными путями. Затем передай отсутствующий файл: результат должен быть понятной ошибкой, а не пустым успешным отчётом.

L2 Жизненный цикл процесса. Добавь генератору выдачу одной строки каждые 10 мс, с принудительным сбросом буфера flush; фактические интервалы могут колебаться. Соедини его каналом pipe с простым регистратором на Python. Останови генератор по SIGINT, дождись EOF и проверь последнюю сохранённую строку. Финальный многопоточный регистратор появится позже.

L3 История настройки. Создай ветку для исправления знака оси y в сценарии наклона. Измени данные и ожидаемый результат вместе, объясни причину в коммите. В отдельном учебном файле конфигурации создай конфликт значения порога, разреши его и повтори нужный тест.

L4 Воспроизведение. В копии проекта восстанови зависимости по lock-файлу и повтори опыт «нет отклика». Команда, версия данных и результат должны совпасть. Снимок VM полезен для восстановления среды, но не заменяет историю проекта и описание опыта.

Связь с дальнейшей робототехникой

Эти навыки нужны, когда датчик или драйвер работает отдельным процессом, опыт идёт на бортовом компьютере, а сбой нужно повторить на ноутбуке. Сохрани папку воспроизводимого опыта и объясни последствия остановки каждого процесса.

Python и диагностика движения

Пять заданий: данные корпуса и сустава

Что изучить перед практикой

Python: функции, dataclass, dict/list, генераторы, with, модули, исключения, argparse, pathlib и JSON. [NumPy](#): массив, shape/dtype, столбцы, маски, abs, mean, max, isfinite, sin/cos и arctan2. [Matplotlib](#): Figure/Axes, несколько линий, подписи, легенда и savefig. Применяй каждую конструкцию к снимку RobotState.

P1 Сгенерируй данные известного движения

Для покоя задай $f=(a_x, a_y, a_z)=(0, 0, 9.81)$, $q_{ref}=q=0$, $i=0.2$. Для статического крена $\phi=15^\circ$: $f=(0, g \cdot \sin\phi, g \cdot \cos\phi)$, $g=9.81$; ϕ для sin/cos переведи в радианы. Сустав неподвижен в обоих опытах. Это идеальные данные с известным ответом.

Для движения сустава задай $q_{ref}=\min(0.8, 0.002 \cdot seq)$, $q=\max(0, q_{ref}-0.02)$, $i=0.2$. В варианте «нет отклика» после $seq=200$ зафиксируй q на его значении при 200, оставив q_{ref} изменяться. Зафиксируй эту границу в конфигурации. Это заданная запись поведения, а не модель физики мотора.

P2 Прочитай и проверь снимок

Реализуй `parse_row` и проверку последовательности отдельно. Читай файл потоково через `with`, сохраняй причины отклонения и инвариант `accepted+rejected=rows_total`. Раздели непредставимое число, пропуск пакета и механически подозрительное поведение: это разные события. Последнее корректное состояние обновляй после всех проверок.

P3 Построй графики данных корпуса

Построй зависимости a_x , a_y , a_z и нормы удельной силы от времени. В статических опытах оцени крен $\phi_{est}=\arctan2(a_y, a_z)$: ожидания 0° и 15° , допуск 0.1° без шума. Добавь к a_y неподвижного корпуса искусственные 2 м/с^2 и покажи ложный наклон. Сделай вывод: [линейное ускорение](#) и вибрации мешают оценивать наклон только по акселерометру; рыскание (y_{aw}) так не определяется.

P4 Выяви отсутствие отклика сустава

Построй q_{ref} и q , затем $e=q_{ref}-q$. Найди $\max|e|$ и среднее $|e|$. Отметь **NO_RESPONSE**, если в окне из 10 снимков с соседними seq каждое $|e|>0.1$ рад, а $\max(q)-\min(q)\leq 0.005$ рад. При разрыве начинай окно заново. Пороги учебные; 10 отсчётов задают выборку, а не точную длительность.

Сопоставь событие с меткой сценария. По одному q нельзя различить застывший энкодер, заклинивший механизм и отключённый мотор: в отчёте пиши «нет ожидаемого отклика». Ток можно нанести рядом как дополнительное наблюдение, но не считать доказательством причины.

P5 Найди разрывы передачи

Удали $seq=40\dots 49$ из чистого файла: из 1 000 снимков останется 990, пропущено 10 номеров seq . Затем испорти одну строку без удаления: `rejected=1`. Пропуск номера среди принятых снимков не доказывает потерю пакета при передаче: строку могли отклонить при проверке. Учитывай эти случаи раздельно.

Готово: JSON-сводка, графики корпуса и сустава, `events.json` с seq и причиной, ручные ожидания для всех опытов. События анализирует Python; C++ и Rust повторяют общий контракт, а не весь анализ.

Современный C++ и управление ресурсами

Время жизни объектов, владение ресурсами и перенос программы разбора данных

Разделы в нужном порядке

- **Сборка программы.** Исходник, заголовок, единица трансляции, объектный файл и линковка. Рассмотрим по одному примеру ошибки компиляции и линковки обработчика датчика. Разбери объявления и определения, `include guards` или `pragma once`.
- **Значения и ссылки.** Инициализация, автоматическое время жизни, область видимости, `const`, передача по значению и по `const T&`. Сравни указатель и ссылку. Разберись, почему ссылка на локальную переменную становится недействительной.
- **[RAII](#).** Конструктор устанавливает корректное состояние, деструктор освобождает ресурс. Файл закрывается и при выходе из функции, и при исключении. Владение обычно выражай объектом, `std::vector` или `std::unique_ptr`; `shared_ptr` применяй только при реальной необходимости совместного владения.
- **Стандартная библиотека.** `std::string`, `std::vector`, `std::array`, итераторы, `range-for`, алгоритмы и `std::optional`. Выясни, когда изменение `vector` делает ссылки недействительными. Изучи `string_view` как невладельющее представление с ограниченным временем жизни.
- **Копирование и перемещение.** Сначала разберись в поведении копирования и перемещения (`copy/move`), затем изучи `std::move`. Он разрешает выбор перемещения, но сам не переносит байты. Для структуры `RobotState` с несколькими числами не создавай сложную иерархию классов.

Практические задания с данными робота

C1 Снимок состояния. Создай `RobotState` из восьми полей контракта. Вынеси парсер, структуру и CLI в отдельные файлы. Передай снимок по значению и `const`-ссылке; объясни, кто владеет им и почему потребитель не должен хранить ссылку на уже переиспользованный буфер датчика.

C2 Приём данных. Перенеси проверку снимков из Python, включая диапазон угла и порядок seq. Прогони покой, движение и испорченные записи. Совпадать должны принятые строки, счётчики, средние `ax/ay/az` и `max|q_ref-q|`; анализ `NO_RESPONSE` остаётся в Python.

C3 Ресурс регистратора. Оберни файл записи в RAII-объект. Сохрани 10 снимков; выйди через `return`, затем через исключение после пятого. Убедись, что объект уничтожен, а ошибка не скрыта. Проверь состояние потока при явном закрытии: уничтожение объекта само по себе не доказывает успешную запись на диск.

C4 Буфер датчика. В отдельном эксперименте сохрани `string_view` на строку IMU, затем переиспользуй исходный буфер. Покажи изменение содержимого; отдельно воспроизведи недействительную ссылку после перераспределения `vector`. Исправь передачу: разбирай строку в `RobotState`, владеющий своими данными, до повторного использования буфера. Сохрани диагностику ASan там, где ошибка обнаруживается.

Готовность ко второму контрольному результату

C++ выдаёт те же счётчики и средние, что ожидаются по контракту. Ты объясняешь владельца каждого объекта и отсутствие висячих ссылок в основном коде. В M1 достаточно C++17; использованные возможности и стандарт зафиксируй в CMake. Шаблонное метапрограммирование, собственные аллокаторы и сложные паттерны проектирования не нужны.

CMake, отладчик и диагностика

Воспроизведи ошибку, найди причину и проверь исправление

Что пройти

В официальном [CMake Tutorial](#) изучи сборку исполняемой программы (executable) и библиотеки (library), разделение подкаталогов, target-команды и Testing and CTest. Пойми связь `add_library`, `add_executable`, `target_link_libraries`, `target_include_directories` и `target_compile_features`. Различай PRIVATE, PUBLIC и INTERFACE на одном примере зависимости; не изучай весь язык CMake.

В проекте нужны библиотека `telemetry`, CLI и цель сборки для тестов. Включи предупреждения `-Wall -Wextra -Wpedantic` для GCC/Clang. Debug используй для отладки, Release - для сравнения скорости. `clang-format` задаёт единый стиль; форматирование не подтверждает корректность программы.

```
cmake -S cpp -B build/debug -DCMAKE_BUILD_TYPE=Debug
cmake --build build/debug
ctest --test-dir build/debug --output-on-failure
cmake -S cpp -B build/release -DCMAKE_BUILD_TYPE=Release
cmake --build build/release
```

Команды рассчитаны на настроенный CMakeLists.txt. В README опиши подключение GoogleTest и пути тестовых данных; зафиксируй версию зависимости для воспроизводимой сборки.

Отладчик по реальному сценарию

D1 Неверные единицы. В учебной версии анализатора передай 15 как радианы вместо 15°. В [GDB](#) поставь точку останова (breakpoint) у проверки `q_ref`, изучи значение и стек, отработай `run`, `next`, `step`, `print`, `backtrace`. Диапазон должен отклонить такую команду; исправь преобразование и добавь тест. Малые ошибки единиц могут попадать в диапазон - одной этой проверки недостаточно.

D2 Повреждённый снимок. В отдельной ошибочной версии C++ обратись к полю `q` после получения строки с недостаточным числом полей. Разбери стек падения и сообщение инструмента диагностики, затем поставь проверку длины до доступа. Сохрани входной файл и тест. `Core dump` изучи на этом примере; если сохранение дампов отключено, отлаживай непосредственно в GDB.

Отдельные сборки с инструментами диагностики

Проверка	Учебный опыт	Что сохранить
ASan и UBSan	Переиспользованный буфер датчика, неверный индекс поля и переполнение целочисленной временной метки.	Фрагмент отчёта, причина, исправленная версия.
TSan	Обновление общего RobotState читателем и монитором без mutex, затем исправление.	Команда, окружение и результат проверки гонки.
Обычные тесты	Повтори набор проверок на исправленной Debug-сборке.	Успешный отчёт и тест на исходную ошибку.

[ASan](#) и [UBSan](#) обычно совместимы; [TSan](#) собирай отдельно. Флаги нужны компилятору и линковщику: `-g`, при необходимости `-fno-omit-frame-pointer`. Не измеряй скорость с диагностикой. Если среда не поддерживает TSan, укажи ограничение и сохрани тесты очереди; проверка гонок остаётся неполной.

Тесты неисправностей робота

Проверь формат CSV и обоснованность выводов о работе

Разделы тестовых инструментов

[pytest](#): assertions, fixtures, tmp_path, raises и parametrize. [GoogleTest](#): TEST, ASSERT/EXPECT, TEST_F и запуск через CTest. В Rust - unit-тесты и cargo test. Общие файлы и expected.json проверяются программой на Python, запускающей три CLI; отдельные тесты Python проверяют диагностику поведения робота.

Сценарий	Ожидание	Что проверяется
S1 Покой и крен	Для безошибочных статических данных крен $0^\circ/15^\circ$ с допуском 0.1° ; норма 9.81.	Оси, единицы и ограничение метода.
S2 Движение	Плавная запись в диапазоне; $\max e \leq 0.02$ рад с числовым допуском; NO_RESPONSE нет.	Различение команды и измерения.
S3 Нет отклика	После фиксации q при seq=200 возникает NO_RESPONSE по правилу P4.	Сигнал неисправности без ложного вывода о причине.
S4 Пропуск записи	Удалить seq=40...49: accepted=990, rejected=0, разрыв принятой последовательности 10.	Пропуск снимков отдельно от плохих строк.
S5 Некорректный ввод	Пустая строка, 7/9 полей, NaN/Inf, 1_000, hex, угол 1.01 рад, повтор seq/t_us.	Причина отклонения; нет обновления принятого состояния.
S6 Исчезновение потока	После нового снимка прошло более 100 мс; FAULT и запрет условной отправки.	Периодическая проверка без новых сообщений.

T1 Рассчитай ожидаемые ответы вручную

Для каждого сценария сохрани конфигурацию генератора и маленький пример с ожидаемым ответом. Углы 0° , 90° и 180° отдельно преобразуй в 0, $\pi/2$ и π ; рабочий сустав всё равно ограничен ± 1 рад. Случаи с $90^\circ/180^\circ$ нужны для проверки функции преобразования, не для выдачи команды суставу.

Проверь ± 1 рад, порог 0.1 рад, пустой файл и CRLF. Для NO_RESPONSE нужны 9 и 10 превышений, затем разрыв seq. Ожидания считай независимо от проверяемого кода.

T2 Намеренно внеси ошибки

Убери перевод градусов в радианы, обнови watchdog при некорректном пакете, разреши NaN в q. Каждый дефект должен выявляться тестом. Исправь код и объясни влияние ошибки на представление о положении ноги или пальца.

T3 Контролируемое время и поток

В тесте watchdog передавай now вручную: последний снимок в 0 мс; tick(100) ещё допустим; tick(101) даёт FAULT. Дубликат в 90 мс не обновляет время приёма допустимого снимка. Не жди настоящие секунды в unit-тестах. Для очереди отдельно нужны EOF, полный буфер, ошибка записи и завершение обоих потоков.

Объём: около 20-25 компактных случаев. В каждом языке проверь парсер; общей программой — эквивалентность CLI; в Python — события датчиков; в C++ — очередь и watchdog. Три полных реализации системы не требуются.

Rust на уровне рабочего инструмента

Понять владение и ошибки через проверку уже известных записей робота

Какие главы Rust Book нужны

Глава 3 - конструкции языка выборочно; глава 4 - ownership, references и borrowing; глава 5 - структуры; глава 6 - enum и match; глава 8 - vector, string и hash map по потребности; глава 9 - Result и обработка ошибок; глава 11 - тестирование. Из глав 12-13 возьми только аргументы CLI, чтение файла и итераторы. Главу 16 о потоках в M1 достаточно просмотреть концептуально.

Разбери, кто владеет String, где значение перемещается, сколько живёт ссылка, почему изменяемое заимствование ограничивает другие обращения и когда нужен Result или Option. Сначала используй владеющие структуры и простые заимствования; сложные аннотации времени жизни отложи.

Минимальный объём реализации

- **R1.** Создай Cargo-проект, структуру RobotState и enum ParseError. Напиши parse_row(&str) -> Result<RobotState, ParseError>. Проверь конечность значений, диапазон сустава и верни понятную причину. Повреждённый снимок не должен становиться состоянием робота; unwrap() не подходит для обработки входного сбоя.
- **R2.** Передай строку снимка IMU по значению, попробуй затем записать ту же String в журнал и прочитай ошибку компилятора. Затем измени параметр на &str. Отдельно создай конфликт изменяемого и неизменяемого заимствования и исправь область использования.
- **R3.** Прочитай журнал ноги или пальца через BufRead, сформируй общую JSON-сводку и коды выхода. Для JSON можно использовать serde/serde_json с зафиксированным Cargo.lock. Не трать время на ручное экранирование JSON.
- **R4.** Через cargo test проверь плохую строку IMU, неверный угол и штатный снимок; подключи общую программу проверки. Выполни cargo fmt --check и cargo clippy, прочитай каждое предупреждение. Для измерений собирай cargo build --release.

Как сравнить Rust и C++ содержательно

Вопрос	C++	Rust
Когда освобождается ресурс	Обычно при завершении времени жизни RAII-объекта.	Обычно при завершении времени жизни владельца через Drop.
Как выражена некорректная строка	Явная структура результата или выбранная политика исключений.	Result с вариантом ошибки и match или оператором ?.
Что происходит с опасным доступом	Часть ошибок выявляется анализом, тестами и инструментами диагностики при запуске.	Многие ошибки заимствования отклоняются компилятором безопасного Rust.

Сохрани два примера диагностики Rust и опыт RAII в C++. Объясни: проверка владения ограничивает способы написания кода, но не отменяет логических ошибок, взаимных блокировок и тестов. Производительность сравнивай по своим измерениям.

Граница готовности. Утилита на Rust проходит общий набор проверок, а ошибки владения ты объясняешь своими словами. Основной обработчик реализуй один раз на C++.

Поток данных и контроль обновлений

Как отличить медленную запись на диск от прекращения потока робота

Модель программы и темы для изучения

Изучи потоки, общую память, mutex, RAIL-блокировку и [condition_variable](#) с предикатом. Поток reader получает снимки, поток writer записывает их через очередь K=64. latest_state хранит принятый снимок и host_received_at. Всё состояние читается и обновляется под одним mutex; файловый ввод-вывод - вне блокировки.

При штатном EOF закрой очередь: установи closed, разбуди потоки, допиши остаток и выполни join. Ошибка writer будит заблокированный reader и завершает опыт с ошибкой. Монитор выполняется в главном потоке. При обработке файла watchdog отключён; в потоке штатный EOF запрещает команды и останавливает монитор до записи остатка очереди.

Q1 Перегрузка регистратора

В режиме **blocking** reader ждёт свободного места; при медленном диске приём новых снимков тоже замедлится. В режиме **drop newest** новая запись не попадает в заполненную очередь, но reader продолжает обновлять latest_state. Так журнал может потерять строки, а монитор - продолжить видеть новые снимки. Отдельно укажи в отчёте пропуски в журнале и задержки приёма, объяснив их причины.

Для контрольной обработки файла используй blocking и отсутствие потерь. Для потокового опыта сравни K=4 и K=64, замедлив writer. Проверь порядок сохранённых seq и ёмкость. После нормального завершения accepted=enqueued+dropped, enqueued=processed. Некорректные строки считаются отдельно; при ошибочном завершении дополнительно укажи число необработанных снимков.

Q2 Контролируй обновления независимо от приёма пакетов

Главный поток примерно каждые 10 мс вызывает tick(now) и проверяет интервал now-last_new_valid_receive. Если прошло **более 100 мс**, записывает FAULT и запрещает условную отправку команды. До первого снимка состояние INIT, команды запрещены; отсчёт идёт от начала приёма. Первый допустимый снимок до таймаута переводит INIT в OK. Дубликаты, старые seq и некорректные строки не обновляют время. Пределы 10/100 мс - учебные настройки, не гарантия реального времени.

В тесте потокового режима останови обновления, оставив reader ждать. Watchdog должен сработать без очередного сообщения. Новый корректный снимок после сбоя не снимает FAULT автоматически: reset разрешён отдельно и только при актуальном принятом снимке. Реальной отправки приводам нет; проверяется переменная command_allowed и запись события.

Q3 Обязательные проверки

- RobotState обновляется целиком: угол и время принадлежат одному снимку. Проверь TSan, если поддерживается.
- Проверь пустой ввод, EOF с остатком, медленный writer и ошибку записи. Для заполнения очереди удержи writer явным сигналом.
- При молчании источника, дубликатах и NaN возникает FAULT по давности допустимого снимка. Границы 100/101 мс проверь управляемыми часами.

Watchdog измеряет давность приёма нового снимка на хосте. Он не доказывает свежесть измерения: данные могли задержаться до reader. Позже понадобятся синхронизация часов и оценка задержки источника.

Повтор опыта и задержки обработки

Повтори сбой и проверь своевременность обновлений монитора

Е1 Повтори обработку записи движения

Сохрани исходные файлы сценариев целиком и отдельный очищенный журнал принятых снимков. Исходные данные нужны для повторения ошибок входа. Числа записывай без потери точности: в C++ используй `max_digits10` для `double` либо сохраняй исходные поля. Сравнивай значения и порядок, а не байты CSV.

Режим `fast` читает файл без ожидания. Повторный анализ сырых данных даёт те же `rejected` и `NO_RESPONSE` с теми же `seq`; очищенного файла - те же принятые состояния.

Выброшенные очередью снимки отсутствуют в очищенном журнале, но доступны в исходном файле сценария. По одному очищенному журналу их восстановить нельзя.

Е2 Проверь временные характеристики

Метрика	Граница измерения и смысл для работа
Интервал источника	Разность соседних <code>t_us</code> ; в чистом сценарии 10 000 мкс. Разрывы отличаются от колебаний интервалов на хосте (джиттера).
Возраст при мониторинге	<code>now_host-last_new_valid_receive_host</code> . По этому значению <code>watchdog</code> замечает молчание канала.
Ожидание и запись	<code>t_done-t_enqueue</code> по одним монотонным часам. Показывает накопление очереди регистратора.
Время трёх CLI	От запуска до завершения на одном файле: включает запуск среды, разбор данных, чтение и сводку.
p50 p95 p99	Отсортируй <code>n</code> задержек; возьми элемент <code>ceil(p*xn)</code> , считая с 1; <code>p=0.50/0.95/0.99</code> . Пустая выборка: <code>null</code> .

Часы: Python [perf_counter_ns](#), C++ [steady_clock](#), Rust `Instant`. Вычитай показания одних часов: нельзя вычитать `t_us` из `now` хоста. Единица «наносекунда» не задаёт точность; VM не гарантирует жёстких сроков.

Е3 Проведи измерения

- Создай один файл из 100 000 снимков движения; зафиксируй SHA-256, `seed` и версии. Сначала проверь эквивалентность трёх CLI. C++/Rust — Release без инструментов диагностики.
- Внешней программой на Python выполни 2 прогрева и 10 замеров каждого CLI, чередуя языки. Сохрани результаты замеров времени, медиану и диапазон. Сравнивай реализации без общего вывода о превосходстве языка.
- Для C++ сравни обычный и замедленный `writer`, `K=4/64`, `blocking/drop newest`. Покажи максимум размера очереди, `dropped`, задержки и события `watchdog`. Для p99 собери не менее 10 000 отдельных задержек.
- В отчёте на 1-2 страницы объясни, сохранилась ли история, получал ли монитор обновления, что ограничило обработку и позволяют ли данные повторить сбой.

Дополнительно: повтор 100-1000 снимков с заданным темпом: `deadline_s=start_s+(t_us-first_t_us)/(1 000 000*speed)`, `speed>0`. Жди заданного абсолютного момента, фиксируй опоздания. В тестах `watchdog` управляй часами.

Расписание первой половины M1

11-23 октября • по 20 часов на блок • задачи одного стенда

Воскресенье - пять практических блоков по 2 часа. Будни - чтение, разбор и небольшие проверки; итоги сохраняй как запись опыта. Даты и суммарная нагрузка соответствуют исходному плану с уточнением на странице 1.

Дата	Ч.	Задача и результат
Вс 11.10	10	Корпус и сустав. Блок 1: среда, репозиторий и L1. Блок 2: оси, единицы, RobotState. Блок 3: генератор покоя, наклона и движения. Блок 4: Python-парсер и сводка. Блок 5: графики P3/P4 и ручная проверка. Результат: первый опыт с графиками.
Пн 12.10	2	Linux и поток. 45 мин пути и перенаправления; 45 мин L2, pipe и EOF; 30 мин проверка остановки. Результат: можно остановить источник и сохранить журнал.
Вт 13.10	2	Python и снимок. 45 мин dataclass/исключения; 45 мин потоковое чтение; 30 мин плохие поля и порядок seq. Результат: схема обновления принятого состояния.
Ср 14.10	2	Данные IMU. 45 мин NumPy/оси; 45 мин покой, крен и ложный наклон при добавлении ускорения; 30 мин график. Результат: понимаешь границы atan2 по акселерометру.
Чт 15.10	2	Версия опыта. 45 мин ветки и история; 45 мин L3, исправление знака оси; 30 мин L4 и lock. Результат: опыт воспроизводится с известными параметрами.
Пт 16.10	2	Диагностика сустава. 45 мин P4 и граница 9/10 отсчётов; 45 мин P5, пропуски и отклонения; 30 мин ожидания. Результат: Python различает выбранные сценарии.
Сб 17.10	0	Отдых; обязательных заданий нет.
Вс 18.10	10	Обработчик C++. Блок 1: компиляция и C1. Блок 2: парсер C2 и сводка. Блок 3: RAll-регистратор C3. Блок 4: буфер C4 и CMake. Блок 5: сравнение с Python. Результат: снимки проверяются и записываются на C++.
Пн 19.10	2	Владение снимком. 60 мин значения, ссылки и const; 40 мин буфер датчика и потребитель; 20 мин выводы. Результат: схема владельцев RobotState.
Вт 20.10	2	RAll и журнал. 60 мин время жизни, сору и move; 40 мин ошибка записи или исключение; 20 мин выводы. Результат: объяснение закрытия файла и проверки ошибок.
Ср 21.10	2	STL и буфер. 45 мин vector/string_view; 45 мин C4 и ASan; 30 мин исправление. Результат: снимок сохраняется при повторном использовании входной строки.
Чт 22.10	2	Сборка стенда. 60 мин цели сборки CMake; 40 мин Debug/Release; 20 мин команды. Результат: отдельно библиотека состояния, CLI, регистратор и тесты.
Пт 23.10	2	Расследование ошибки. 45 мин GDB; 45 мин D1/D2, единицы и неверная длина снимка; 30 мин тест. Результат: вход, причина, исправление и доказательство.

На 23 октября: обе реализации одинаково принимают и отклоняют снимки; регистратор сохраняет значения. В отчёте видны оси, единицы и предел сустава. Расхождения исправь до переноса контракта в Rust.

Расписание второй половины M1

25 октября - 3 ноября • 35 часов на сбои, поток и проверку

Дата	Ч.	Задача и результат
Сб 24.10	0	Отдых.
Вс 25.10	10	Сбои и Rust. Блок 1: scenarios/expected.json и S1-S5. Блок 2: pytest/GoogleTest и T1/T2. Блок 3: Cargo, RobotState и Result. Блок 4: валидатор Rust. Блок 5: общая программа проверки и исправления. Результат: три CLI проверяют один контракт.
Пн 26.10	2	Доказательства. 45 мин параметризация; 45 мин 9/10 превышений, разрыв и неверные единицы; 30 мин T2. Результат: понятно, какой тест ловит каждый сбой.
Вт 27.10	2	Владение Rust. 60 мин владение и заимствование; 40 мин строка датчика для анализа и журнала; 20 мин объяснение. Результат: исправлены два конфликта владения.
Ср 28.10	2	Некорректное состояние. 45 мин Result/Option; 45 мин NaN, угол вне диапазона и Cargo tests; 30 мин RAII/ownership. Результат: плохой снимок не обновляет состояние.
Чт 29.10	2	Обмен между потоками. 60 мин mutex/condition_variable; 40 мин Q1 и закрытие; 20 мин схема. Результат: известны владельцы latest_state и политика переполнения.
Пт 30.10	2	Контроль обновлений. 45 мин Q2 и часы; 45 мин T3, границы 100/101 мс; 30 мин план измерений. Результат: проверяемая логика watchdog до интеграции потоков.
Сб 31.10	0	Отдых.
Вс 01.11	10	Интеграция. Блок 1: reader/writer, очередь и завершение. Блок 2: latest_state, монитор и FAULT. Блок 3: Q3, перегрузка и E1 replay. Блок 4: E2/E3, сырые измерения. Блок 5: отчёт и чистая сборка. Результат: кандидат telemetry-core v1.0.
Пн 02.11	3	Разбор и резерв. 60 мин выводы о пропусках, задержках и причинах; 60 мин инструкция опыта; 60 мин один проблемный сценарий. Используется один дополнительный час программы.
Вт 03.11	2	Проверка стенда. 45 мин объяснение сценариев без подсказки; 45 мин чек-лист; 30 мин итоги. Результат: навыки и список невыполненных критериев.

Как уложить интеграцию в бюджет

До последнего воскресенья подготовь снимки, последовательный регистратор, общий набор проверок и проверенную tick(now). Соедини их: watchdog в главном потоке, reader/writer в рабочих. Отдельная система задач или реактор не нужны.

Обязательны один сустав, оси IMU и файлы и графики. Сначала сократи расширения: шум, график тока, rased replay, дополнительные размеры входа. Полный робот, физическая симуляция, ROS, PID и фильтр ориентации идут позже. Непройденные базовые тесты отмечай как незавершённость M1.

Итоговый разбор: почему журнал отстаёт, пока монитор получает новые снимки? Подтверди ответ счётчиками и временными метками.

Литература: маршрут по задачам

Литература по программированию и обязательный первый разбор механизма

План и определения - на русском; часть литературы - на английском. Читай указанный фрагмент перед задачей и сразу проверь идею кодом. Покупать четыре книги и читать их целиком не требуется. Основное чтение выделено; к справочнику обращайся при затруднениях.

Основное чтение

- 1. Scott Chacon, Ben Straub - Pro Git, 2-е изд.** [Русский текст книги](#). Разделы 2.1-2.5: создание репозитория, запись изменений, история, отмена и удалённые репозитории; 3.1-3.2: ветвление и слияние. Для L3/L4: ветка исправления оси, конфликт порога, версия воспроизводимого опыта.
- 2. Steve Klabnik, Carol Nichols и сообщество Rust - The Rust Programming Language. Rust Book.** Главы 4-6 и 9: владение и ссылки, структуры, enum/match и ошибки - R1/R2. Из 11-13: тесты, CLI и чтение файла и итераторы - R3/R4. Главы 3 и 8 выборочно по синтаксису и коллекциям; 16 только обзорно. Async, unsafe и FFI отложи.
- 3. Anish Athalye, Jon Gjengset, Jose Javier Gonzalez - The Missing Semester of Your CS Education, MIT, курс 2020.** [Конспекты курса](#). «Course Overview + The Shell», «Command-line Environment», «Debugging and Profiling»: перенаправления, процессы, диагностика и измерения для L1/L2, D1/D2, E3. По сборке и зависимостям - фрагменты «Metaprogramming».

Справочник C++: конкретные разделы

- 4. Bjarne Stroustrup - A Tour of C++, 3-е изд.** [Автор и оглавление](#). §1.5-1.7: время жизни, const, указатели и ссылки; §3.2 и 3.4: отдельная компиляция, аргументы; §6.2-6.3: copy/move и ресурсы - C1-C3. §10.2-10.3 и 12.2: string, string_view, vector - C4. §18.2-18.4: потоки, общие данные, ожидание - Q1-Q3. Книга рассчитана на опыт программирования; начинай после P1/P2. Используй подмножество C++17; modules и coroutines здесь не нужны.

Документация для конкретного шага

[Python Tutorial](#): Control Flow, Data Structures, Modules, Input/Output, Errors and Exceptions, Classes, Virtual Environments - P1/P2. NumPy/Matplotlib и тестовые инструменты открываются по ссылкам на страницах задач. [REP-103](#) (Units, Axis Orientation) и [REP-145](#) (Data Sources) поясняют оси и акселерометр; установка ROS для чтения не нужна.

Основы движения: что именно описывает программа

Звено - часть механизма, которую в модели считаем твёрдой. **Сустав** задаёт допустимое относительное движение звеньев: вращательный - угол, поступательный - смещение. **Степень свободы** - одно независимое локальное движение. У закреплённой ноги с незамкнутой цепью звеньев с тремя независимыми вращательными суставами их три; число приводов и число степеней свободы могут различаться.

Конфигурация q определяет взаимное положение звеньев; у закреплённого одиночного шарнира это угол. Для механической модели второго порядка состояние включает также скорость: $x=(q, \dot{q})$, где $\dot{q}$ - угловая скорость в рад/с. Одинаковая поза при разной скорости - разные состояния. RobotState M1 - снимок наблюдений, а не полное физическое состояние.

Кинематика связывает координаты, скорости и ускорения без расчёта сил. **Динамика** объясняет изменение движения через массу, инерцию, силы и моменты. **Управление** выбирает воздействие для достижения цели. Угол из обратной кинематики ещё не доказывает, что мотор выполнит движение. Подробнее: M8, M9, M10.

Читать перед P1: Kevin Lynch, Frank Park, Modern Robotics, [§2.1: конфигурация и свободы](#) и [§2.2: суставы](#). Разбери определения, типы суставов и схему механизма; общая формула подвижности - в M5.

Основные термины M1

Определения для схемы данных, владения ресурсами и обработки потока

Телеметрия (telemetry). Передача измеренных величин и диагностических событий для наблюдения за системой; сама по себе не управляет движением.

Снимок состояния (state snapshot). Согласованный набор значений одного шага; в M1 восемь полей имеют общую метку `t_us`.

Система координат (frame). Начало и ориентированные оси, относительно которых заданы компоненты вектора; здесь *x* вперёд, *y* влево, *z* вверх.

Радиан (radian). Единица плоского угла: отношение длины дуги к радиусу. Полный оборот равен 2π рад; $\theta_{\text{рад}} = \theta_{\text{град}} \cdot \pi / 180$.

Удельная сила (specific force). Показание идеального акселерометра: $f = a - g$; *a* - ускорение движения, *g* - гравитационное ускорение, выраженные в одной системе координат. В покое при *z* вверх $f_z \approx +9.81 \text{ м/с}^2$.

Крен (roll). Поворот корпуса вокруг продольной оси *x*; статическая оценка $\text{atan2}(a_y, a_z)$ искажается линейным ускорением и не позволяет определить рыскание (*yaw*).

Ошибка положения (position error). Разность запрошенного и измеренного углов $e = q_{\text{ref}} - q$, в радианах; величины описывают команду и наблюдение соответственно.

Метка времени (timestamp). Время события по выбранным часам; `t_us` источника и время хоста нельзя вычитать без согласования шкал.

Монотонные часы (monotonic clock). Шкала, не идущая назад; подходит для интервалов внутри одного источника времени, но не задаёт календарную дату.

Инвариант (invariant). Условие, которое должно сохраняться в допустимых состояниях программы; например, $\text{accepted} + \text{rejected} = \text{rows_total}$ после обработки файла.

Владение (ownership). Ответственность за время жизни ресурса. Владелец обеспечивает освобождение; невладеющая ссылка не продлевает время жизни ресурса.

RAII. Связывание ресурса со временем жизни объекта C++: освобождение выполняет деструктор. Уничтожение объекта не доказывает успешную запись.

Заимствование (borrowing). Доступ Rust по ссылке без передачи владения, ограниченный временем жизни и правилами совместимости изменяемого и неизменяемого доступа.

Висячая ссылка (dangling reference). Ссылка на объект, время жизни которого завершилось либо адрес стал недействительным; обращение через неё нарушает корректность программы.

Мьютекс (mutex). Средство взаимного исключения: защищает общий участок состояния от одновременного конфликтующего доступа при согласованном использовании всеми потоками.

Гонка данных (data race). Несинхронизированные конфликтующие обращения потоков к одной памяти, когда хотя бы одно - запись; в C++ приводят к неопределённому поведению.

Обратное давление (backpressure). Замедление производителя, когда потребитель не успевает: в режиме blocking поток reader ждёт, пока в полной очереди освободится место.

Контроль обновлений (watchdog). Периодическая проверка давности приёма нового допустимого снимка по часам хоста; молчание вызывает FAULT без очередного сообщения.

Задержка (latency). Время между двумя заданными событиями: например, enqueue и окончанием записи. Границы и источник часов задаются до измерения.

Квантиль p99. Значение, ниже или равно которому лежит примерно 99% наблюдений; здесь берётся элемент $\text{ceil}(0.99 \cdot n)$ упорядоченной выборки, считая с 1.

Воспроизведение (replay). Повторная обработка сохранённого входа; сырые строки нужны, чтобы воспроизвести также отклонения, отсутствующие в очищенном журнале.

Хеш SHA-256. 256-битный отпечаток содержимого для проверки совпадения файлов; сам по себе не подтверждает автора и подлинность происхождения.

Проверь себя: почему в покое $a_z \approx g$; почему неизменный *q* не устанавливает причину сбоя; почему dropped, rejected и пропуски seq - разные счётчики?

Что купить и установить в M1

Подготовка до L1, C1 и R1; время настройки уже включено в 75 часов

Оборудование и покупки

Имеется: 1 MacBook Pro M4, терминал и имеющийся 3D-принтер; профиль принтера выбирается по точной модели позже. Печать в M1 не нужна. **Обязательные покупки: нет.** IMU, MCU, моторы, аккумуляторы, робот и GPU не нужны: данные создаёт генератор на Python. Книгу по C++ можно взять в библиотеке; покупать её необязательно.

Установить до первого практикума: 11 октября

На macOS ARM: редактор, например [VS Code для Apple Silicon](#), и [UTM](#). Рабочая гостевая ОС - [Ubuntu 24.04 ARM64](#); выбери образ arm64 и виртуализацию ARM, а не x86-эмуляцию. Если такая Ubuntu уже настроена, используй её. Выдели VM ресурсы по объёму памяти и диска Mac, проверь свободное место и отзывчивость до опытов.

В Ubuntu ARM64: Git, терминал и командная оболочка, ripgrep; [uv](#) и Python; зависимости numpy, matplotlib, pytest в одном проекте. Зафиксируй Python и зависимости в pyproject.toml/uv.lock, команды в README. Не ставь пакеты опыта в системный Python. Графики сохраняй в PNG, чтобы опыт работал и без GUI.

До C1: 18 октября; до R1: 25 октября

C++ в Ubuntu: GCC/G++ либо Clang с C++17, CMake, средство сборки make/Ninja, GDB, clang-format; GoogleTest для тестов с зафиксированной версией. Пакеты Ubuntu: [build-essential](#), [cmake](#), [gdb](#), [clang-format](#); запиши выбранный компилятор и версии. ASan/UBSan - отдельная Debug-конфигурация; TSan отдельно и только при поддержке окружением.

Rust в той же Ubuntu: [rustup](#), rustc, Cargo, rustfmt и Clippy. Зафиксируй toolchain и Cargo.lock. Для R3 добавляй serde/serde_json по необходимости. Нативные macOS-сборки - дополнительная проверка; контрольные измерения трёх CLI делай в одной Ubuntu.

Проверка установки и необязательные дополнения

Запиши архитектуру гостя (aarch64) и версии. Выполни Git commit; через uv запусти импорт NumPy/Matplotlib/pytest и сохранение графика; собери и запусти C++17-пример через CMake/CTest, остановись в GDB в точке останова; выполни cargo test, cargo fmt --check, cargo clippy. В чистой копии восстанови зависимости по lock и повтори L1/L4. Среда готова к задаче после успешной проверки соответствующего инструмента.

Необязательно: Docker только если понадобится изоляция сверх VM; в M1 не нужен. ROS 2, Gazebo и GPU-стек откладываются до своих модулей. Чип Apple M4 не поддерживает CUDA. VM не гарантирует жёсткое реальное время; графику и USB проверяй перед будущими аппаратными опытами. Доставка последующих покупок не входит в учебные часы.

Основа: roadmap.pdf, физическая стр. 3 - календарь; стр. 19-21 - цели M1. Схема RobotState, сценарии, учебные пороги и приёмка разработаны для подробного плана. Установки и проверка среды - стр. 15.

Первый расчёт движения: до генератора R1

Плоское звено $L=0.20$ м закреплено шарниром в начале координат: x вправо, y вверх; q отсчитывается от $+x$ против часовой стрелки. Конец: $p=(L \cos q, L \sin q)$. Для $q=0$ получишь $(0.20; 0)$ м, для $q=0.50$ рад - $(0.17552; 0.09589)$ м. Масса в этом расчёте не нужна: это прямая кинематика.

В блоке 2 11 октября замени часть разбора RobotState: 30 мин на определения и схему, 30 мин на расчёт и график. Сохрани mechanism.md и рисунок; проверь $\|p\|=L$ и $q=-0.50$. Почему по q и L нельзя узнать момент мотора? CSV, предел $[-1;1]$ рад и 75 часов сохраняются.